REDIS · DISCOVERY & INVENTORY

Find every Redis-compatible database on your network.

Redis Discovery is a read-only discovery and inventory tool for Redis-compatible databases. Scan authorized network ranges or known host lists, identify endpoints, collect inventory, and export results — from either the `rscan` CLI or a local Web UI.

CLI rscan scan · credential-scan · serve

Web UI Discover · Results · Export

Safe no writes · no brute-force

What's new

The newest capabilities at a glance — the rest of the guide covers everything in full detail.

- Credential Scan now accepts an `.ini` file, not just CSV — see rscan credential-scan and the Credential Scan page.
- The Results page has a new **INI with credentials** button for re-attaching credentials to an export without retyping them — see Exports.
- CSV export is now a complete dump of every field the JSON API returns, not just what the results table shows.
- The results table's Memory column shows used memory versus the configured maximum, not just used memory.

01

What Redis Discovery does

Redis Discovery helps answer questions such as:

- Which Redis-compatible instances are reachable on this network?
- Which hosts require authentication?
- Which discovered endpoints are the same backing database reachable through multiple addresses?
- What product / version / role / memory / keyspace / module / cluster / TLS certificate details are visible from a safe read-only probe?
- Can I export the results for analysis or downstream tools such as `osstats`?

Safe by default

- It performs TCP connection attempts against the targets and ports you specify.
- On open ports it uses a short read-only Redis command sequence such as authentication, `PING`, `INFO`, `MODULE LIST`, and `CLUSTER INFO`.
- It does not write data, change configuration, alter cluster state, or brute-force passwords.
- Credentials are used only for the scan or request that needs them and are not persisted.

> ⚠ **Important:** Only scan networks, hosts, and Redis-compatible databases that you are authorized to test.

02

Installation and build

Prerequisites

- Node.js 22 or later
- npm

Build from source

```
git clone https://github.com/bcooper-redis/redis-discovery.git
cd redis-discovery
npm install
npm run build
```

The build compiles TypeScript into `dist/` and copies the Web UI assets into `dist/web/public/`.

Run without linking

```
node dist/cli/index.js --help
```

Install the local `rscan` command

From the project directory:

```
npm link
rscan --help
```

After linking, use `rscan` in the examples below.

03

CLI overview

The CLI has three main commands:

```
rscan scan              # Scan CIDRs, IPs, or hostnames
rscan credential-scan   # Scan explicit host/port rows with per-row credentials
rscan serve             # Start the local Web UI
```

You can inspect help at any time:

```
rscan --help
rscan scan --help
rscan credential-scan --help
rscan serve --help
```

Progress and summaries are written to **stderr**; results are written to **stdout**. This lets you redirect or pipe result data without mixing in progress messages:

`rscan scan -c 10.0.0.0/24 --json > results.json`

04

rscan scan

Use `rscan scan` to discover Redis-compatible databases across one or more targets. Targets can be CIDR ranges, single IP addresses, or hostnames.

Basic syntax

```
rscan scan [options]
```

Options

OPTION	DEFAULT	DESCRIPTION
`-c, --cidr <target>`	Auto-detected local subnets	Target to scan. Can be a CIDR, IP, or hostname. Repeat the flag for multiple targets. Hostnames resolve to IPv4/A records. Add `:port` to a target to scan that target on a specific port instead of the shared `--port` list.
`-p, --port <ports>`	6379	Ports to scan. Accepts a single port, comma-separated list, or ranges such as `6379,6380-6385`.
`-t, --timeout <ms>`	1000	Per-connection timeout in milliseconds. Values below 1 are coerced to at least 1. Invalid values fall back to 1000.
`--concurrency <n>`	100	Maximum number of concurrent connection attempts. Values below 1 are coerced to at least 1. Invalid values fall back to 100.
`--tls`	Off	Attempt TLS first. If the TLS handshake fails, Redis Discovery automatically falls back to a plain connection.
`--tls-skip-verify`	Off	Skip TLS certificate verification. Useful for self-signed certificates or private CAs during authorized testing.
`--username <user>`	None	ACL username. Requires `--password`.
`--password <pass>`	None	Password used only for this scan. It is not logged, printed, or persisted.
`--json`	Off	Print results as formatted JSON instead of a table.

Target formats

```
# CIDR range
rscan scan -c 10.0.0.0/24
# Single IP
rscan scan -c 10.0.0.42/32
# Hostname (resolved via DNS, IPv4/A records)
rscan scan -c redis-cache.internal.example.com
# Multiple targets
rscan scan \
  -c 10.0.0.0/24 \
  -c 10.1.5.25/32 \
  -c redis-cache.internal.example.com
# Target-specific port
rscan scan \
  -c redis-a.internal.example.com:6379 \
  -c redis-b.internal.example.com:6380
```

If a hostname resolves to multiple IPv4 addresses, every resolved address is scanned. When a target includes `:port`, that target is scanned only on that port instead of the shared `--port` setting.

If you omit `-c`, Redis Discovery auto-detects local non-loopback subnets, capped at a `/24` per interface. Use this carefully and only on networks you are authorized to scan.

Port specifications

```
rscan scan -c 10.0.0.0/24                    # default port 6379
rscan scan -c 10.0.0.0/24 -p 6380            # one alternate port
rscan scan -c 10.0.0.0/24 -p 6379,6380,16379 # explicit list
rscan scan -c 10.0.0.0/24 -p 6379,6380-6385  # ranges
```

TLS examples

```
# Try TLS first, fall back to plain
rscan scan -c 10.0.0.0/24 -p 6379,6380 --tls
# TLS endpoints with self-signed certificates
rscan scan -c 10.0.0.0/24 -p 6380 --tls --tls-skip-verify
```

Authentication examples

```
# Password-only Redis
rscan scan -c 10.0.0.0/24 -p 6379 --password 'correct-horse-battery-staple'
# Redis ACL username and password
rscan scan \
  -c 10.0.0.0/24 \
  -p 6379 \
  --username inventory-reader \
  --password 'correct-horse-battery-staple'
```

`--username` requires `--password`. Supplying a username without a password is a usage error.

JSON output

```
rscan scan -c 10.0.0.0/24 -p 6379,6380 --json > redis-discovery-results.json
```

Because progress goes to stderr and JSON goes to stdout, the redirected file contains only JSON.

Table output

Without `--json`, results are printed as a table:

COLUMN	MEANING
HOST	Discovered host or IP address.
PORT	Discovered Redis-compatible port.
TLS	yes or no, based on how the endpoint was reached.
PRODUCT	Product family such as Redis OSS, Redis Enterprise, Valkey, KeyDB, or unknown.
VERSION	Redis-compatible server version, when available.
AUTH	Authentication state: open, required, authed, wrong pw, or error.
ROLE	Redis role, such as master, replica, or unknown, when inventory is available.
MEMORY	Used memory vs. max memory. `no limit` means maxmemory is unset.
LATENCY	Probe latency in milliseconds.

```
HOST       PORT  TLS  PRODUCT    VERSION  AUTH      ROLE    MEMORY              L
─────────  ────  ───  ─────────  ───────  ────────  ──────  ──────────────────  ─
10.0.0.12  6379  no   redis OSS  7.2.4    open      master  12.4 MB / no limit  4
10.0.0.13  6380  yes  valkey     8.0.0    required  –       –                   9
```

Exit behavior

A completed scan exits `0`, including the case where no Redis instances are found. Usage or input errors exit `1` — for example an invalid CIDR, invalid port specification, `--username` without `--password`, scan expansion above the safety cap, or no CIDR supplied and no local subnet detected.

Common scan recipes

```
rscan scan -c 127.0.0.1/32                                  # localhost only
rscan scan -c 192.168.1.0/24                                # subnet, default port
rscan scan -c 192.168.1.0/24 -p 6379,6380,6381,16379    # Redis + TLS + alternate

# Redis Enterprise-like env where a DB may appear on many endpoints
rscan scan -c 10.20.0.0/24 -p 12000-12050 --json > redis-enterprise-scan.json
# then group JSON results by inventory.runId to detect duplicate endpoints

# Clean JSON while watching progress; capture both separately
rscan scan -c 10.0.0.0/24 --json > results.json
rscan scan -c 10.0.0.0/24 --json > results.json 2> scan.log
# Slower network timeout
rscan scan -c 10.0.0.0/16 -p 6379 -t 3000 --concurrency 200
```

05

rscan credential-scan

Use `rscan credential-scan` when you already have a known list of hosts and each host may have its own username/password. This differs from `rscan scan`, which applies one optional credential pair to all discovered endpoints.

Basic syntax

```
rscan credential-scan -f known-hosts.csv [options]
```

Options

OPTION	DEFAULT	DESCRIPTION
`-f, --file <path>`	Required	CSV file containing `host,port,username,password` (a header row is allowed and skipped automatically), or an `.ini` file in the same format Export INI produces. The format is chosen automatically from the file extension.
`-t, --timeout <ms>`	1000	Per-connection timeout in milliseconds.
`--concurrency <n>`	100	Maximum number of concurrent connection attempts.
`--tls`	Off	Attempt TLS first, with plain fallback on TLS handshake failure.
`--tls-skip-verify`	Off	Skip certificate verification for self-signed or privately issued certificates.
`--json`	Off	Print results as formatted JSON instead of a table.

CSV format

```
host,port,username,password
10.0.0.12,6379,,open-passwordless-probe
10.0.0.13,6379,inventory_reader,s3cr3t
10.0.0.14,6380,default,"p@ss,word"
```

- `host` is required; `port` must be valid.

- `username` may be blank; `password` may be blank — blank credentials mean the row is probed without attempting authentication.
- If a password contains a comma, wrap it in double quotes.
- Malformed rows are skipped with warnings on stderr; valid rows still run. If no valid rows remain, the command exits with an error.

INI input format

You can also point `-f` at an `.ini` file in the exact format Export INI produces. This is meant for a round trip: run a scan, use Export INI to get a skeleton with host, port, and tls filled in, add username/password for the hosts you have credentials for, then feed that same file back in.

- One `[host:port]` section per target. The section header itself is not parsed — only the `host =` and `port =` lines inside it are.
- `username` and `password` may be blank, same meaning as a blank CSV field.
- Lines starting with `;` or `#` are comments and are ignored, including the commented-out `ca_cert/client_cert/client_key` placeholders Export INI writes — this tool has no mTLS support.
- Malformed sections are skipped with warnings on stderr, same as a malformed CSV row.

Credential scan examples

```
rscan credential-scan -f known-hosts.csv
rscan credential-scan -f known-hosts.csv --json > credential-scan-results.json

# TLS and skip certificate verification
rscan credential-scan \
  -f known-hosts.csv \
  --tls \
  --tls-skip-verify \
  --json > credential-scan-results.json

# Tune timeout and concurrency for sensitive/slower endpoints
rscan credential-scan \
  -f known-hosts.csv \
  -t 3000 \
  --concurrency 50
```

Use `rscan scan` when...

- You need to discover unknown Redis-compatible endpoints.
- You want to scan CIDRs, IPs, or hostnames.
- You have zero credentials, or one pair that applies broadly.

Use `rscan credential-scan` when...

- You already have a host inventory.
- Each host has its own username/password.
- You want per-host authentication results in a single pass.

06

rscan serve

Use `rscan serve` to start the local Web UI.

OPTION	DEFAULT	DESCRIPTION
`--port <port>`	3000	HTTP port for the Web UI.
`--host <host>`	localhost	Host/interface to bind.

```
rscan serve                          # open http://localhost:3000
rscan serve --port 8080              # open http://localhost:8080
rscan serve --host 0.0.0.0 --port 3000
```

⚠ Only bind to `0.0.0.0` if you understand that the Web UI and its scan/authenticate endpoints may be reachable from your network. The Web UI does not provide its own access control.

07

Web UI guide

Start the UI with `rscan serve`, then open the printed URL (usually `http://localhost:3000`). The UI has five pages:

- Discover
- Credential Scan
- Results
- Settings
- About

Only one scan can run at a time. Starting another scan while one is scanning or paused returns a conflict until the current scan is finished or stopped.

08

Discover page

Use Discover to run a standard discovery scan. It is the UI equivalent of `rscan scan`.

Targets field

Enter one target per line. Supported formats:

```
10.0.0.0/24
192.168.1.50
redis.example.com
redis.example.com:6380
10.0.0.0/24:6380
```

- Blank target field means auto-detect local non-loopback subnets.
- CIDRs, single IPs, and hostnames are supported; hostnames resolve through DNS and all resolved IPv4 addresses are scanned.
- A target ending in `:port` is scanned only on that port instead of the shared Ports field.

Upload CSV button

Populate the Targets field from a CSV file (`host` or `host,port`, header optional).

- The file is read in the browser — the raw file is not uploaded to the server.
- Parsed targets replace the current Targets field; `host,port` rows convert to `host:port`.
- The form applies the same timeout, concurrency, TLS options, and optional credentials to every target. For per-host credentials, use the Credential Scan page.

Ports, Timeout & Concurrency

- **Ports** — shared port list for targets without their own `:port` (e.g. `6379,6380-6385`, `12000-12050`).
- **Timeout** — per-connection ms. Start at 1000 for nearby LANs; 2000–5000 for slower networks, VPNs, or firewalled paths. Very low values may miss live instances.
- **Concurrency** — max simultaneous attempts. 100 is a reasonable default; lower it for fragile networks or very large scans; raise cautiously for fast, trusted internal networks.

TLS & certificate options

- **Attempt TLS first** — tries a TLS handshake before plain Redis; on failure it falls back to plain. Certificate details can be collected even when Redis-level auth is required.
- **Skip certificate verification** — does not reject self-signed or untrusted certificates. Use for self-signed certs, private CA deployments, and lab/test environments. Avoid when you need to validate trust chains.

Credentials section

- Optional scan-wide Username (ACL) and Password, used for this scan only and applied to every target.
- Credentials are not stored, logged, or persisted. Leave both blank to scan anonymously.
- Use Credential Scan if each host has different credentials.

Start Scan begins the scan and navigates to Results. If a scan is already running or paused, the request is rejected until it finishes or is stopped.

09

Credential Scan page

Use Credential Scan when you have a known host list and each row may have its own username/password.

CSV or INI upload

```
host,port,username,password
10.0.0.12,6379,,
10.0.0.13,6379,inventory_reader,s3cr3t
10.0.0.14,6380,default,"p@ss,word"
```

- A header row is allowed; username and password may be blank for anonymous probing; comma-containing passwords must be double-quoted.
- The file is parsed in the browser — the raw CSV is not uploaded; parsed values are sent when the scan starts. Each row's credentials are used only for that row.
- You can also upload an `.ini` file — the same format Export INI produces. Iterate: run a scan, Export INI, fill in credentials in a text editor, then upload here. Format is chosen from the extension.

Timeout and Concurrency behave as on the Discover page; the same TLS-first and skip-verify checkboxes apply. **Start Credential Scan** stays disabled until a valid CSV is loaded. Results appear on the shared Results page. Credential scans are not restartable from Results because per-row credentials are not retained.

10

Results page

The Results page shows live scan state, controls, discovered inventory, duplicate detection, authentication actions, and exports.

Banners & status

The **target banner** shows what is being scanned (with an *auto-detected* badge for local subnets). The **status banner** can show: `idle`, `scanning`, `paused`, `done`, `error`, `stopped`, plus progress and elapsed time while active.

Scan controls

- **Pause** — stops scheduling new attempts; already-open connections finish naturally.
- **Resume** — continues a paused scan.
- **Stop** — stops immediately and keeps results found so far.
- **Restart** — re-runs the last Discover scan with the same non-sensitive options. Restarted scans run anonymously (credentials are never persisted); use per-row Authenticate afterward. Not shown after a Credential Scan.
- **Refresh** — refreshes displayed state and results.

Duplicate detection

Redis Discovery detects likely duplicate endpoints when multiple results share the same Redis `run_id` — commonly when a Redis Enterprise database is reachable through multiple cluster-node proxy endpoints, or a local instance is found through multiple interfaces.

- A duplicate banner appears; **Hide duplicates** is checked by default.
- Checked: each duplicate group collapses to a single representative row. Unchecked: every endpoint shows with duplicate indicators.
- Export CSV/INI/XLSX follows the current duplicate setting.
- CLI behavior differs: the CLI table always prints the full uncollapsed list plus a duplicate warning; JSON carries raw `runId` values so you can group them yourself.

Results table columns

COLUMN	MEANING
Host / Port	Host or IP address, and Redis-compatible port.
Product / Version	Product family (Redis OSS, Redis Enterprise, Valkey, KeyDB, unknown) and server version when available.
TLS	Whether the final successful probe used TLS.
Auth	Authentication state: open, required, authed, wrong password, or error-like states.
Role / Mode	Master, replica, or unknown; standalone, cluster, or sentinel when available.
Cluster / Replicas / Replicating From	Cluster state summary, connected replica count, and master host/port for replicas.
Memory / Keys	Used vs. configured max memory (no limit when unset), and key count summary.
Modules / OS / Uptime / Latency	Loaded module names, OS from Redis INFO, server uptime, probe latency.
Run ID	Redis run_id, used for same-database detection.
TLS Cert / Cert Expires	Certificate subject/issuer/trust summary and expiration for TLS endpoints.
Action	Authenticate button when applicable.

Authenticate button

Each row can offer an Authenticate action to provide credentials for one host after discovery. The dialog takes an optional ACL Username and a required Password. Submitting authenticates to that host and updates its row with full inventory on success. Credentials are used for that request only and are not retained for restart or future scans.

TLS certificate info without Redis credentials

For TLS endpoints, Redis Discovery can collect certificate metadata from the handshake even when Redis-level auth is required — Subject, Issuer, Valid from, Valid to, Self-signed

status, Trust status, and SHA-256 fingerprint. This is often the only useful information for an auth-required TLS endpoint when you lack credentials.

11

Exports

The Results page supports three export formats. Every export respects the current Hide duplicates setting.

CSV export

The most complete flat result set for spreadsheets, scripts, or ad hoc analysis. CSV is a complete dump of every field the JSON API returns — beyond the visible columns it includes whether auth was required, connected client count, max/peak/total system memory, full per-replica detail, full per-database keyspace detail, full per-module detail (version and path), full cluster info (enabled, slots assigned, known nodes, size), and the certificate's validFrom date and SHA-256 fingerprint.

INI export

An osstats-compatible `config.ini`. One `[host:port]` section per result with host, port, and tls filled in. username and password are left blank because Redis Discovery does not retain credentials.

INI with credentials

A second button next to Export INI for when you want the credentials back in the file. It warns that the resulting file contains plaintext credentials, then lets you pick a CSV or `.ini` file — typically one you already filled in from an earlier Export INI. Its username/password values are matched to the on-screen results by host and port and merged into a freshly generated INI.

- Entirely client-side: the file you pick, the matching, and the generated download all happen in the browser — no request is sent to the server.
- The plain Export INI button and the HTTP API are unaffected — they still always return blank credentials.
- Only entries matching a host:port in the current results get credentials filled in; everything else in the file is ignored.

XLSX export

An Excel workbook shaped like osstats' `OSStats.xlsx` output. Sheet name `ClusterData`. Includes fields Redis Discovery can know from a single point-in-time probe; it does not fabricate throughput or command-stat deltas that require long-running sampling.

12

Settings page

Settings stores non-sensitive defaults that pre-fill the Discover page.

SETTING	MEANING
Default ports	Default shared port list for Discover scans.
Default timeout (ms)	Default per-connection timeout.
Default concurrency	Default maximum concurrent connections.
Attempt TLS first	Default TLS-first setting.
Skip certificate verification	Default TLS certificate verification behavior.

Save Defaults stores defaults in the browser; **Reset to Defaults** clears custom defaults and restores built-in ones. Defaults are stored only in the browser's localStorage and are not sent to the server until you start a scan. Credentials are not stored in Settings.

13

HTTP API quick reference

The Web UI uses the local HTTP API exposed by `rscan serve`. You can also script against it.

Start a standard scan

```
curl -sS -X POST http://localhost:3000/api/scan \
  -H 'Content-Type: application/json' \
  -d '{
    "cidrs": ["10.0.0.0/24"],
    "ports": [6379, 6380],
    "timeoutMs": 1000,
    "concurrency": 100,
    "tls": true,
    "tlsSkipVerify": false
  }'
```

Start a credential scan

```
curl -sS -X POST http://localhost:3000/api/credential-scan \
  -H 'Content-Type: application/json' \
  -d '{
    "targets": [
      { "host": "10.0.0.12", "port": 6379, "username": "inventory_reader", "passw
      { "host": "10.0.0.13", "port": 6379 }
    ],
    "timeoutMs": 1000,
    "concurrency": 100,
    "tls": false,
    "tlsSkipVerify": false
  }'
```

Results & scan control

```
curl -sS http://localhost:3000/api/results | jq .

curl -sS -X POST http://localhost:3000/api/scan/pause
curl -sS -X POST http://localhost:3000/api/scan/resume
curl -sS -X POST http://localhost:3000/api/scan/stop
curl -sS -X POST http://localhost:3000/api/scan/restart
```

Authenticate one host and update inventory

```
curl -sS -X POST http://localhost:3000/api/inventory \
  -H 'Content-Type: application/json' \
  -d '{
    "host": "10.0.0.12",
    "port": 6379,
    "username": "inventory_reader",
    "password": "s3cr3t"
  }'
```

Export current results

```
curl -sS http://localhost:3000/api/export/csv  -o redis-discovery.csv
curl -sS http://localhost:3000/api/export/ini  -o config.ini
curl -sS http://localhost:3000/api/export/xlsx -o OSStats-like.xlsx

# Export with duplicates collapsed
curl -sS 'http://localhost:3000/api/export/csv?excludeDuplicates=true'  -o redis-
curl -sS 'http://localhost:3000/api/export/ini?excludeDuplicates=true'  -o config
curl -sS 'http://localhost:3000/api/export/xlsx?excludeDuplicates=true' -o redis-
```

14

JSON result structure

`--json` and `/api/results` expose structured discovery results.

Top-level result fields

FIELD	MEANING
`host` / `port`	Host or IP address and port number.
`tls`	Whether TLS was used.
`product`	Product identifier: redis, valkey, keydb, enterprise, or unknown.
`version`	Server version, when known.
`authRequired`	Whether Redis-level auth is required.
`anonymousStatus`	Anonymous probe status: open, auth_required, unreachable, or error.
`authenticatedStatus`	Auth attempt status: authenticated, auth_failed, or not_attempted.
`latency`	Probe latency in milliseconds.
`inventory`	Full inventory object when available; null when it could not be collected.
`tlsCertificate`	TLS certificate object for TLS endpoints, or null for plaintext.

Inventory fields

FIELD	MEANING
`redisVersion`	Version reported by Redis INFO.
`mode`	standalone, cluster, or sentinel.
`os` / `uptimeSeconds`	OS reported by Redis INFO; uptime in seconds.
`role` / `replication`	master, replica, or unknown; replica/master relationship details.
`memory`	Used/max/peak/system memory and eviction policy.
`keyspace`	Per-database key and expiry counts.
`modules`	Loaded modules with name/version/path.
`clusterInfo`	Cluster state, slot, and node details when available.
`runId`	Redis run_id, useful for duplicate detection.
`connectedClients`	Connected client count, when available.

TLS certificate fields

FIELD	MEANING
`subject` / `issuer`	Certificate subject and issuer.
`validFrom` / `validTo`	Start and end of validity period.
`selfSigned`	Whether subject and issuer match.
`trusted`	Whether the chain validated against Node.js's CA store.
`fingerprint256`	SHA-256 fingerprint.

Example jq queries

```
# List discovered endpoints
jq -r '.[] | "\(.host):\(.port) tls=\(.tls) product=\(.product) auth=\(.anonymous
# Show endpoints requiring auth
jq -r '.[] | select(.authRequired == true) | "\(.host):\(.port)"' results.json
# Group by run_id to find likely duplicate endpoints
jq -r '
  group_by(.inventory.runId // "")[]
  | select(length > 1 and .[0].inventory.runId != null)
  | "runId=\(.[0].inventory.runId) endpoints=\([.[] | "\(.host):\(.port)"] | joir
' results.json
# Show TLS certificate expirations
jq -r '.[] | select(.tlsCertificate != null) | "\(.host):\(.port) expires=\(.tlsC
```

15

Docker

```
# Build the image
docker build -t redis-discovery .
# Run the Web UI (open http://localhost:3000)
docker run --rm -p 3000:3000 redis-discovery
# Run a one-shot scan
docker run --rm redis-discovery scan -c 10.0.0.0/24 -p 6379,6380
```

The container image runs the Web UI by default using `serve --host 0.0.0.0 --port 3000` so the UI is reachable through the published Docker port.

Docker networking caveats

- `127.0.0.1` inside the container is the container, not your host.
- Auto-detected subnets may be Docker bridge networks, not your LAN.
- Explicit external CIDRs usually work through Docker's outbound NAT.

```
# On Linux, use host networking to see the host network namespace
docker run --rm --network host redis-discovery scan -p 6379
```

To reach a service on the host without host networking, use `host.docker.internal` where supported, but resolve it to an IP first because scan targets are CIDR/IP/hostname forms.

16

Troubleshooting

No Redis instances found

Possible causes: wrong target range, wrong port list, firewall or routing block, timeout too short, or Redis not listening on the scanned interface. Try:

```
rscan scan -c 10.0.0.0/24 -p 6379,6380 -t 3000
nc -zv 10.0.0.12 6379
```

Live Redis reports as not Redis

Possible causes: a non-Redis service on that port, a restrictive ACL preventing required read-only commands, or a proxy/gateway returning unexpected protocol data. Confirm the account can run at least the inventory commands needed for discovery.

Username without password error

```
# Invalid
rscan scan -c 10.0.0.0/24 --username inventory_reader
# Use
rscan scan -c 10.0.0.0/24 --username inventory_reader --password 's3cr3t'
```

Hostname cannot be resolved

If any hostname target fails to resolve, the scan is rejected. Redis Discovery uses IPv4/A records for hostname expansion. Check:

```
nslookup redis.example.com
dig redis.example.com A
```

TLS scan falls back to plain unexpectedly

A TLS handshake failure causes automatic plain fallback. Common causes: the port is not TLS-enabled, the certificate is untrusted and `--tls-skip-verify` was not used, or a middlebox/proxy interfered. Try:

```
rscan scan -c redis.example.com:6380 --tls --tls-skip-verify
```

Scan target too large

Redis Discovery enforces a host-count safety cap. Split large scans into smaller CIDRs:

```
rscan scan -c 10.0.0.0/24
rscan scan -c 10.0.1.0/24
rscan scan -c 10.0.2.0/24
```

Web UI cannot reach the server

```
rscan serve --port 3000 --host localhost   # then open http://localhost:3000
```

Build cannot find HTMX assets

```
npm install
npm run build
```

Security and responsible use

Redis Discovery is intentionally read-only, but it is still a network scanner. Treat it accordingly.

- Get explicit authorization before scanning.
- Prefer narrow CIDRs and explicit host lists.
- Start with conservative concurrency on shared networks.
- Avoid putting passwords directly in shell history when possible.
- Do not expose the Web UI broadly without an external access-control layer.
- Use `--tls-skip-verify` only when you intentionally accept untrusted/self-signed certificates.

Safer credential handling

```
# Avoid typing the password literally into shell history
read -s REDIS_PASSWORD
rscan scan -c 10.0.0.0/24 --username inventory_reader --password "$REDIS_PASSWORD
unset REDIS_PASSWORD

# For credential CSV files
chmod 600 known-hosts.csv
rscan credential-scan -f known-hosts.csv --json > results.json
```

Practical workflows

A — Quick local discovery

```
rscan scan   # auto-detect local subnets on an authorized network
```

B — Controlled subnet discovery with JSON artifact

```
rscan scan \
  -c 10.30.4.0/24 \
  -p 6379,6380-6385 \
  -t 2000 \
  --concurrency 100 \
  --json > redis-discovery-10.30.4.json
```

C — TLS inventory with self-signed certificates

```
rscan scan \
  -c 10.30.4.0/24 \
  -p 6380 \
  --tls \
  --tls-skip-verify \
  --json > redis-tls-inventory.json
```

D — Known hosts with per-host credentials

```
# known-hosts.csv
host,port,username,password
10.30.4.12,6379,inventory_reader,s3cr3t1
10.30.4.13,6379,inventory_reader,s3cr3t2
10.30.4.14,6380,default,"p@ss,with,commas"

chmod 600 known-hosts.csv
rscan credential-scan -f known-hosts.csv --json > credentialed-inventory.json
```

E — UI-assisted discovery and export

```
rscan serve
```

Then: open `http://localhost:3000` → Discover → enter targets and ports → choose TLS options → Start Scan → review duplicates on Results → Authenticate individual rows if needed → Export CSV, INI, or XLSX.

F — Generate an osstats-compatible INI

Run a UI scan (or start one via the API), review Results, leave Hide duplicates checked for one endpoint per detected backing database, then click INI export. Or via API:

```
curl -sS 'http://localhost:3000/api/export/ini?excludeDuplicates=true' -o config.
```

G — Iterating on a batch of credentials

- Run a scan (Discover, or Credential Scan with whatever credentials you have) and open Results.
- Click Export INI for a skeleton with host/port/tls filled in, or re-export with **INI with credentials** to keep credentials you already tried.
- Fill in username/password for a batch of hosts in a text editor, then run `rscan credential-scan -f that-file.ini` (or upload it on the Credential Scan page).
- Review the Auth column — authed rows are done; wrong pw or required rows still need attention.
- Click INI with credentials, pick that same file, and you get a fresh INI with your credentials still there — edit just the rows that failed. Repeat until everything you have credentials for shows authed.

19

Field reference: authentication states

The table and JSON expose two related concepts: **Anonymous status** (what happened when probing without credentials, or before supplied credentials) and **Authenticated status** (what happened when credentials were supplied).

AUTH VALUE	MEANING
`open`	Endpoint allowed inventory without authentication.
`required`	Redis-level authentication is required and no successful credentials were used.
`authed`	Authentication succeeded and inventory could be collected.
`wrong pw`	Authentication was attempted but failed.
`error`	Probe resulted in an error state.

20

Field reference: duplicate detection

Redis `run_id` identifies a running Redis server process. Redis Discovery uses it to identify likely duplicate endpoints.

```
[
  { "host": "10.0.0.11", "port": 12000, "inventory": { "runId": "abc123" } },
  { "host": "10.0.0.12", "port": 12000, "inventory": { "runId": "abc123" } }
]
```

These rows likely represent the same running database reachable through two endpoints.

- **In the UI** — Hide duplicates checked: one representative row remains. Unchecked: all endpoints remain visible.
- **In the CLI** — table output shows all rows and prints a duplicate warning; JSON output includes all rows and the raw `runId` values.

21

Recommended defaults

```
# Most authorized LAN scans
rscan scan -c 10.0.0.0/24 -p 6379,6380 -t 1000 --concurrency 100
# Slower or more sensitive networks
rscan scan -c 10.0.0.0/24 -p 6379,6380 -t 3000 --concurrency 25
# TLS-heavy environments
rscan scan -c 10.0.0.0/24 -p 6380 --tls --tls-skip-verify
# Audit/export workflows
rscan scan -c 10.0.0.0/24 -p 6379,6380 --json > redis-discovery-results.json
```

22

Limitations and non-goals

Redis Discovery is a point-in-time discovery and inventory tool. It does **not**:

- Continuously monitor Redis endpoints.
- Brute-force passwords.
- Persist credentials.
- Change Redis configuration or write data to Redis.
- Fabricate performance counters that require long-running sampling.
- Provide built-in authentication for the Web UI.

Use it for safe discovery, inventory, credential validation, duplicate endpoint detection, and export — not for load testing, continuous observability, or unauthorized scanning.